

Artificial Deterministic Intelligence: Probabilistic Reasoning, Deterministic Authority

Branden Laskowski
Cortex Agentics Global

Version 1.0 | 9 October 2026 | Architectural position and bounded evidence synthesis

Probabilistic reasoning. Deterministic authority.

Abstract

Language models can propose useful actions without being reliable authorities over the systems that execute them. We define Artificial Deterministic Intelligence (ADI) as an architectural approach in which probabilistic models provide bounded inference while identity, authority, memory admission, policy, execution rights, state transitions, verification, and release are governed by deterministic systems external to the model. Determinism refers to specified control decisions under explicit state and inputs; it does not imply deterministic generation, correct policies, truthful memory, or universal safety. We present a layered reference model and separate conceptual requirements from observations in a governed research evidence ledger. In a first-party synthetic memory benchmark of 30 designs repeated three times per arm, scoped 8B and 35B configurations each met the recorded acceptance contract on 90/90 jobs, while scoped 4B and broader-context 35B configurations accepted 72/90 and 46/90; 43 broader-context responses were truncated under the fixed output budget. A separate eight-task context-admission diagnostic retained PARTIAL overall parity despite matching the corrected context condition. Historical memory tests provide bounded implementation evidence; worker-construction performance records with incomplete artifact lineage remain excluded. These observations motivate, but do not establish, the complete ADI architecture. The proposal synthesizes established work on reference monitors, capability security, formal methods, runtime assurance, neuro-symbolic systems, and governed agents. Its contribution is an explicit lifecycle-wide authority contract, an evidence taxonomy, and falsifiable evaluation questions. Independent replication, comparative evaluation, and validation of the full enforcement boundary remain open.

Keywords: artificial deterministic intelligence; agent governance; bounded inference; capability security; governed memory; runtime verification; model substitution; evidence provenance.

1 Problem statement

A language-model application often combines several decisions in one conversational exchange: what the user meant, what information is relevant, which identity is acting, whether an action is allowed, and whether its result may be retained or released. These decisions have different failure consequences. A plausible answer can be useful even when uncertain. A plausible permission is not sufficient authorization to change a record, invoke a privileged tool, or disclose an artifact. An architecture therefore needs an explicit account of which outputs are proposals and which components can confer authority.

This distinction is older than modern language models. Complete mediation and least privilege are central security principles [1,2]. Runtime assurance separates an advanced controller from an independently enforced safety boundary [5]. Recent language-model systems likewise combine proposals with external verifiers, policy enforcement, information-flow restrictions, and runtime constraints [10–14]. The question addressed here is how to state that separation consistently across an artificial worker's lifecycle, including memory admission and release, and how to evaluate it without turning a successful language task into a claim of general system safety.

ADI is proposed as an architectural vocabulary and research program. The name does not identify a new class of deterministic language model or establish priority over external-control architectures. The intended unit of analysis is a complete deployed application: its admitted identities, policy versions, memory transitions, tool boundary, evidence records, and release decisions, together with its inference providers. A named component or a prompt that describes these responsibilities is insufficient. The responsibilities must be enforced at the relevant effects.

The paper makes three contributions. First, it specifies a control contract that distinguishes inference from authority across the lifecycle. Second, it organizes existing evidence by what it can and cannot establish, retaining negative and incomplete findings. Third, it proposes falsifiable tests that could distinguish a useful ADI implementation from an ordinary agent with descriptive guardrails. The evidence is retrospective and first-party. This manuscript introduces no new benchmark runs, rescoring, security proof, or independent replication.

2 Definition and scope of determinism

2.1 Definition

Artificial Deterministic Intelligence is an architectural approach in which probabilistic models provide bounded inference while identity, authority, memory admission, policy, execution rights, state transitions, verification, and release are governed by deterministic systems external to the model. The operational phrase is: Probabilistic reasoning. Deterministic authority.

Bounded inference means that the model receives a task and permitted context, produces a proposal or answer within a stated contract, and cannot enlarge its own privileges through its generated content. The bound includes what the surrounding system can actually enforce. A request to obey a rule is not an enforcement mechanism. Identity is a system-issued principal and its bindings; it is not a model's assertion about who it is. Memory admission is a state-changing decision about whether information may be retained, at what status, and for which scope. It is separate from whether the information sounds convincing.

Deterministic authority means that a specified decision function produces the same control result for the same fully specified inputs, policy version, and relevant state. Those inputs must include, or explicitly abstract, time, revocation state, external observations, and event order. If two concurrent requests race, determinism cannot be inferred from a deterministic parser alone. The implementation must define how the relevant order is established and how changes between authorization and execution are handled.

The requirement is conditional on an explicit trusted computing base. It is not a claim that hardware, administrators, databases, networks, or sensors are infallible. Hashes establish identity of inspected bytes under ordinary cryptographic assumptions; they do not prove the bytes describe true events. Repeatable policy evaluation can reliably enforce a mistaken rule. A verifier can be sound for a narrow property and irrelevant to the user's real objective.

2.2 A minimal abstract contract

Let S be the authoritative state, P a versioned policy, I an authenticated principal and task binding, X the recorded external inputs, and q a model proposal. A control function $D(S, P, I, X, q)$ returns deny, hold, or a bounded authorization together with reasons and required checks. An execution transition T applies only when that authorization is valid for the effect being attempted. The pair D and T is an abstract specification, not a description of proprietary implementation.

A proposed invariant is that every protected effect has a valid authorization linked to the actual principal, action, object, policy, and relevant state. A second is that a denied or held proposal cannot cause that effect through an alternate path. A third is that a model response cannot itself mint authority, admit canonical

memory, revise the controlling policy, or release an artifact. These are requirements to prove or test in a concrete system; the evidence in Section 5 does not prove them universally.

A useful substitution property is conditional: if the proposal, principal, policy, external inputs, and relevant state are identical, the authorization decision should not depend on which model produced the proposal, unless model identity is explicitly part of policy. Different models may produce different proposals and therefore legitimately induce different decisions. ADI consequently does not promise identical action trajectories across models. It seeks a stable authority boundary around those trajectories.

2.3 What deterministic authority does not mean

Deterministic generation concerns repeatability of model output. A fixed seed or temperature setting is neither required nor sufficient for the authority contract above. An application could use variable generation behind a deterministic authorizer, or repeatable generation with unrestricted credentials. Only the former arrangement addresses the proposed separation; its security still depends on actual mediation and correct policy.

Logical correctness, factual truth, and permission are also distinct. A permitted action may be strategically poor. A correct statement may be inadmissible for a particular recipient. An admitted memory record may later be corrected. An answer evaluator may accept a response without establishing that any real-world action occurred. For learned classification near the boundary, ADI requires an explicit treatment of uncertainty: the classification may be evidence for a decision, but it cannot silently replace the authoritative policy or create its own privileges. For the strong authority claim used here, a learned semantic verdict cannot be the sole authorizing predicate for a protected effect. It must remain a proposal checked against independently authorized, mechanically evaluable conditions, or be held for an explicitly authorized human decision. If policy intentionally accepts a learned classification as decisive, the deployment has a probabilistic authorization dependency and must state that narrower boundary. A deterministic wrapper does not remove it.

3 Architecture and layer model

Table 1 describes responsibilities, not a mandatory process topology. Several layers may reside in one service. A distributed implementation must still preserve the stated bindings. No internal algorithms, private schemas, or deployment layout are needed to express these obligations.

Table 1. Conceptual ADI responsibilities and observable contracts.

Layer	Responsibility	Observable contract
Identity and construction	Establish principal, task, role, and permitted capabilities before activation	Generated role text cannot create a principal or grant
Policy and authority	Evaluate explicit, versioned rules over authenticated inputs	Same fully specified decision inputs produce the same result
Memory admission and context	Separate candidate, admitted, contextual, and unresolved information	Retrieval respects scope and status; admission has provenance
Probabilistic inference	Interpret, plan, summarize, or propose within supplied bounds	Outputs remain proposals until applicable checks succeed
Execution and state	Mediate protected effects and coordinate state changes	Authorization is checked for the actual effect and relevant state
Verification and evidence	Check defined properties and preserve outcomes	Failed, held, and partial results remain distinguishable

Layer	Responsibility	Observable contract
Release and accountability	Bind a dissemination decision to an artifact and destination	Preparation does not imply permission to publish or disclose

The threat model treats model proposals, retrieved text, tool responses, and task-supplied content as potentially mistaken or adversarial. It assumes the authority mechanism, its installed policy, principal authentication, and protected effect adapters are within the trusted computing base. Administrative compromise of that base is outside the proposed contract; unmediated application paths are conformance failures, not excluded attacks. The paper does not prove that any implementation meets these assumptions.

The architecture has two kinds of feedback. An inference result can be returned for correction without changing authority. Separately, an authorized operator can revise policy or state through a governed transition. Conflating these loops creates a route by which persuasive generated text changes the rules used to judge itself. Policy updates are therefore themselves protected effects, with versioning and revocation semantics appropriate to the application.

3.1 Worker construction before inference

A worker is conceptually a principal plus a task-bound configuration: identity, permitted actions, relevant procedures, context access, and completion requirements. Descriptive knowledge about an application is distinct from authority to control it. Loading a procedure can help interpretation without supplying credentials or granting an execution right. Construction before inference makes this distinction explicit, but it is a design requirement rather than evidence that a particular worker is competent.

An ADI implementation should be able to reject an unauthorized effect even if a model claims to have a new role or reports a fabricated approval. It should also be able to replace the inference provider while retaining the externally established principal and its state. The first property concerns enforcement; the second concerns lifecycle continuity. Neither follows merely from a successful prompt or a model-substitution score. Worker-construction efficiency and continuity-preserving live replacement remain research questions in this paper because the available canonical evidence does not establish them.

3.2 Memory as governed state

Retrieval and admission answer different questions. Retrieval determines which retained information may be provided for a task. Admission determines what can become authoritative state, with whose provenance, at what confidence or disposition, and under which correction policy. Unresolved information must remain representable without being treated as established. Context and interpretation may be valuable while having a different status from verified observations.

Corrections should preserve the relationship to earlier records. Otherwise, an apparent success rate can improve simply because failures disappear. Temporal validity requires a rule for when a record expires or is superseded, and when that rule is evaluated. Correct behavior during one ingestion does not establish correct behavior after time advances. Memory scope, temporal correctness, provenance, and answer quality are separate evaluation dimensions.

3.3 Execution, verification, and release

A complete effect boundary covers tools, durable memory changes, outbound messages, and publication as appropriate to the application. Checking a proposed action after an irreversible effect is only detection. Likewise, an authorization evaluated against stale state may no longer be valid at execution time. Implementations need an atomic or otherwise justified relationship between permission checks and protected transitions, with failure behavior specified for retries, outages, and partial completion.

Verification should be named by the property checked. A schema validator establishes conformance to a structure. A source hash establishes byte identity. A policy checker establishes an allowed decision relative to its policy. A scientific reviewer evaluates whether observations support prose claims. None can substitute for all the others. Release combines its own artifact, destination, and disclosure constraints; it is not automatically inherited from a successful experiment or a finished PDF.

4 Relationship to prior and adjacent work

4.1 Reference monitors, capabilities, and policy engines

The reference-monitor tradition supplies the strongest antecedent to external authority: access checks should be completely mediated, protected from tampering, and sufficiently tractable for analysis [1,2]. Capability security supplies explicit, limited authority and delegation mechanisms [3]. ADI adds no new security primitive to these traditions. Its contribution is to demand that language-model lifecycle claims identify which of these obligations are actually enforced, including memory and release.

Policy languages and engines already separate application behavior from authorization decisions. XACML, Open Policy Agent, and Cedar illustrate mature approaches to expressing and evaluating access rules [7–9]. A policy engine alone does not guarantee that all effects pass through it, that its inputs are authentic, or that an application's policy is correct. An ADI claim should identify this composition gap rather than describing external policy evaluation as an end-to-end guarantee.

4.2 Formal methods and runtime assurance

Formal methods provide ways to state invariants and verify implementations relative to assumptions [4]. Execution-monitoring approaches formalize which security policies can be enforced by observing and potentially stopping executions [6]. Runtime verification more generally concerns whether observed executions satisfy a stated specification. Simplex-style assurance separates an advanced controller from a safety mechanism capable of limiting its behavior [5]. These approaches anticipate the essential relationship between flexible problem solving and externally enforced constraints.

ADI is not a replacement for proof, model checking, or runtime monitors. It offers places to apply them. For example, one might verify that denied actions cannot reach a protected interface, then test whether that interface is the only path to the resource. A proof about an abstract transition system remains conditional on the fidelity of its implementation and environment. The seL4 verification work illustrates the stronger evidence and explicit assumptions involved in a concrete verification claim [22]. Presenting ordinary deterministic code as though it had been formally verified would overstate the proposal.

4.3 Neuro-symbolic systems and model-plus-verifier architectures

Neuro-symbolic research combines learned representations with symbolic structure or reasoning [15]. Its scope is broader than authority: a symbolic component can support inference without holding any execution privilege. Conversely, a conventional authorization service can enforce policy without symbolic reasoning about an entire task. ADI overlaps this literature where symbolic constraints govern admissibility, but does not require a particular representation of knowledge or a new neural architecture.

LLM-Modulo explicitly combines language-model proposals with external critics or verifiers, including a distinction between formal and softer feedback [10]. This is close conceptual prior art. ADI's proposed emphasis is lifecycle coverage: the same separation should be articulated for worker activation, memory admission, protected effects, and release. That broader statement is a synthesis whose usefulness must be evaluated; it is not evidence that prior verifier architectures lack lifecycle protections.

4.4 Governed agents and information-flow control

Recent systems make novelty claims about external agent control especially difficult. CaMeL separates control and data and uses explicit tracking and enforcement to constrain tool interactions [11]. AgentSpec provides runtime-enforced constraints for agent behavior [12]. Progent controls privileges and mediates tool use with a programmable policy approach, including mechanically checked policy narrowing and explicit approval for privilege expansion [13]. A broader design-pattern literature examines how agent structure, capability restriction, and isolation can reduce prompt-injection exposure [14].

These works should be comparison baselines, not background citations used to imply that external enforcement is new. ADI does not claim to outperform them. A substantive future contribution would demonstrate a clearly specified lifecycle property, a useful conformance test, or a reproducible comparison that these baselines make possible. Prompt injection remains a relevant adversarial input, but authority separation alone does not establish confidentiality, availability, semantic correctness, or protection against every authorized but harmful action.

4.5 Retrieval, persistent agents, and model substitution

Retrieval-augmented generation and memory-oriented agent systems externalize information that need not reside in model weights [16,17]. Reasoning-and-acting frameworks interleave model decisions with environmental interaction [18]. AIOS also places agent services, including memory and access control, outside individual agent applications [21]. These establish that external state and tool use are not distinctive inventions of ADI. The proposed distinction concerns the authority to admit, retrieve, execute, and release, not the mere presence of a database or a tool loop.

The companion manuscript Externalized Intelligence and Model Substitution analyzes the same developer benchmark lineage [19]. It provides supporting interpretation and protocol context, not an independent replication or a second source of empirical authority. This umbrella paper uses governed ledger records for its observations [20]. It intentionally declines some stronger diagnoses and worker-cost claims when they are not established by the selected canonical claim mappings.

5 Evidence and analysis

5.1 Source of truth and evidence classes

The empirical source of truth is the governed Research Evidence Ledger, read for the ADI paper scope on 9 October 2026. The frozen snapshot contains 18 evidence identities: 12 canonical and six held. A later finalization read contained two additional held CI candidate notices; neither was admitted as empirical support. The original 18 records and their admission states remained unchanged. [ADI-P1, ADI-P2] These are record identities, not 18 independent experiments. Canonical admission concerns provenance and exact artifact-backed observations. Experimental result status is a separate field, so a canonical record can legitimately contain HOLD or PARTIAL findings. EVIDENCE_MAP.md supplies the full evidence_id, exact claim tag, source claim identifier, relevant artifact hash, and permitted interpretation for each claim below.

We use four labels. A conceptual claim defines an architectural obligation or interpretation. A recorded observation reports a canonical ledger value. A derived quantity is arithmetic over identified recorded values. A hypothesis is a prediction requiring new evidence. Held records are reported only as exclusions and preserved history, not as verified measurements. Literature claims are attributed to their primary sources. No aggregate success rate is computed across studies with different tasks, scorers, or evidence classes.

All observations are developer evidence from one implementation lineage. Canonical admission does not confer independence, establish complete experimental documentation, or prove the conceptual architecture. The ledger's admission decisions were not independently reimplemented here. Live retrieval and mapping verify that this manuscript refers to the admitted observations; they do not establish the truth of every experimental premise or the correctness of the original scorer.

5.2 Model substitution and context selection

Table 2 reports the admitted acceptance counts without rescoring. The M1 predecessor remains visible even though M1-V2 supersedes it in the default current-claim view. Historical retrieval was required to inspect it. The labels 4B, 8B, and 35B abbreviate the recorded model configurations gemma3:4b, qwen3:8b, and qwen3.6:35b; they are not controlled measures of intelligence or active computation. [ADI-E1–ADI-E5]

Table 2. Accepted jobs divided by attempted jobs under each recorded contract.

Study	Scoped 4B	Scoped 8B	Scoped 35B	Broader or full 35B
M1 historical predecessor	—	—	3/4	4/4
M1-V2	—	—	4/4	4/4
M2	3/4	—	—	4/4
M3	—	3/4	—	4/4
M4	11/12	12/12	12/12	9/12
M5	72/90	90/90	90/90	46/90

M1–M3 use the recorded full-reprompt condition; M4–M5 use their broader-control condition. They are not interchangeable baselines. M1-V2 records a different scorer version from M1. M2 and M3 remain HOLD results. We do not reinterpret an acceptance failure as proof that a forbidden real-world action executed, and do not retroactively award credit on semantic grounds. [ADI-E1, ADI-E2, ADI-E3]

The strongest bounded substitution observation is M5: both scoped 8B and scoped 35B accepted 90/90 jobs under the recorded contract. Scoped 4B accepted 72/90. The broader 35B arm accepted 46/90 and completed 47/90. Not all four arms met acceptance, and M5's aggregate result remains HOLD. The ledger identifies 8B as the smallest tested scoped model passing that workload; it does not establish a threshold between 4B and 8B or equivalence on unseen work. [ADI-E5]

The registered M5 claim fields contain arm aggregates. Supplemental inspection of the ledger's hash-bound raw-trial artifact supplies the method: 30 synthetic task designs repeated three times in each arm, a 2,048-token output budget, and the frozen m5-structured-answer-v1 scorer. The 360 stored rows are observations on repeated designs, not 360 independent tasks. Six required-HOLD designs repeat three times per arm; all 18 jobs in the scoped 4B arm fail the stored scorer's disposition requirement. The broader 35B arm records 43 truncated responses and one additional completed rejection. These are preserved statuses and scores, not a new evaluation of model answers. [ADI-E13]

The M5 raw artifact also retains complete=false despite completedRounds=3 and allPlannedJobsAttempted=true. We therefore describe the attempted workload and its outcomes without relabelling the run complete or successful. These supplemental protocol and diagnostic observations are bound to the same canonical evidence ID and raw artifact hash; they are not separately registered claim fields, and the claim-mapping endpoint does not validate them. The evidence audit records the exact inspection pointers and counting rules. The ledger records a local Apple M4 Pro with 48 GB memory for M4/M5; this is benchmark metadata, not a replicated hardware study. [ADI-E4, ADI-E5, ADI-E13]

M5's recorded active wall totals are 828,638.781 ms for scoped 8B and 900,074.233 ms for scoped 35B. Their difference is 71,435.452 ms, or 7.937% of the scoped 35B total, rounded from the ledger values. This descriptive comparison excludes a general claim about end-to-end setup, monetary cost, energy, or hardware

efficiency. Model family, tokenizer, output behavior, context selection, and ordering can affect it. Equal finite-workload acceptance is not a non-inferiority analysis. [ADI-E6]

5.3 Context parity and governed memory

M6 records PARTIAL overall parity. Both compared arms accepted 8/8 tasks while their input totals differed: 3,812 versus 12,137 tokens. A follow-up, M6-P2, records context-admission parity after correction, with the corrected condition accepting 8/8 and using 3,812 input tokens. The before-correction result remains preserved. M6-P2 still records PARTIAL overall parity, with unresolved differences involving authorization, temporal handling, scope/context behavior, and metadata semantics. [ADI-E7, ADI-E8]

The important inference is limited: answer acceptance can agree while context and system behavior differ. Correcting one context condition does not demonstrate equivalence of every memory rule or deployment path. The diagnostic uses an eight-task subset and one recorded 8B model configuration; it is not a cross-model continuity experiment, a general temporal-correctness result, or a demonstration of production readiness. The proprietary mechanics underlying the differences are unnecessary to state this limitation and are not disclosed here.

Two historical records provide additional memory evidence. A repository test-suite record reports 1,052 passed, zero failed, and ten skipped in its full-suite summary. Its classification is historical test-suite evidence. A separate in-memory reference verification records PASSED for its reported checks. The latter's generated timestamp uses a deterministic verification clock and is not established as a wall-clock execution time. Neither record measures present production reliability or independently reproduces the M-series workload. [ADI-E9, ADI-E10]

5.4 Worker construction and excluded performance evidence

Final A1-V2 records four tasks per mode, a pass rate of one in each mode, and zero parameter reduction in its frontier object. It is a same-model comparison admitted through a committed byte-identical fixture. It does not demonstrate cross-model substitution, improved general reasoning, or worker-construction efficiency. Earlier A1 and the initial A1-V2 attempt remain held for incomplete verified artifact lineage. [ADI-E11; ADI-H1]

A2, A3, and A4 include candidate observations about worker context cost and benchmark quality, but their admission remains held. Their preservation is valuable: a positive candidate must meet provenance requirements just as a negative one must. This manuscript therefore reports no verified worker-construction speedup from these records, does not import their numerical results from repository prose, and does not erase their failed or incomplete history. Worker construction remains part of the conceptual architecture and a gap in the canonical empirical case. [ADI-H1]

5.5 Endurance and negative results

A canonical synthetic runtime-endurance record contains a 300-concurrency phase configured for 30 minutes and a 450-concurrency phase configured for 60 minutes. The recorded phase totals sum to 469,191 completed assignments and 4,691,910 operations. These counts establish the bounded observation in that endurance record. They do not measure governed-memory correctness, useful language work per second, or production capacity. The separate combined 700-job/90-minute recollection has no verified matching raw artifact and remains held. [ADI-E12; ADI-H2]

The negative evidence is part of the result rather than an appendix to a success narrative. Canonical M1, M2, M3, M4, and M5 retain incomplete acceptance; M6 and M6-P2 retain PARTIAL system parity. Held A-series records remain excluded even when candidate summaries are favorable. The empirical case supports narrower observations about selected configurations and incomplete parity, not a claim that deterministic authority makes every task succeed. [ADI-E1–ADI-E8; ADI-H1]

6 What the evidence supports

The evidence supports a limited design motivation: changing the context condition and model configuration can coexist with the same finite-workload acceptance, and answer-level agreement can coexist with material system differences. It is reasonable to study external authority, context admission, and inference as separate dimensions. It is not reasonable to infer from these observations that the complete proposed authority boundary is enforced under arbitrary adversarial input.

The contribution of ADI at this stage is conceptual integration. The experiments do not isolate the causal effect of the entire architecture. There is no controlled comparison of an otherwise identical system with and without every ADI responsibility. There is no universal claim of model independence, no demonstration that external memory contains all of a worker's competence, and no claim that symbolic control transfers intelligence from a model into an operating system.

A useful deployment evaluation would therefore have separate verdicts for task quality, authority enforcement, memory correctness, lifecycle continuity, and reproducibility. A release decision would depend on the requirements of the specific application. Collapsing these verdicts into one label would make a successful answer look like a security guarantee or a successful policy check look like scientific correctness.

7 Threats to validity

Construct validity is the principal risk. Acceptance is defined by a frozen task contract, not by an all-purpose measure of intelligence, safety, or factual competence. A scorer can reject an acceptable paraphrase or accept a response that omits an untested requirement. The current manuscript preserves recorded classifications and does not resolve those possibilities by retrospective regrading. Determinism is likewise a property of a specified transition or decision, not a favorable adjective for a system.

Internal validity is limited by a shared development lineage and by configurations that differ in multiple respects. Model families, inference behavior, budgets, selected context, caching, and host conditions can affect acceptance and time. The broader-context arm changes the task burden and has many non-completions, so it cannot establish general superiority of a smaller model. A before-and-after correction on known tasks does not substitute for a randomized or independent holdout evaluation.

Statistical validity is limited by small pilots, repeated designs, and an unspecified deployment population. We report descriptive counts and one arithmetic time comparison. We do not calculate a pooled success rate, claim statistical equivalence, or use a perfect finite sample to infer a zero failure probability. Test-suite counts also do not measure independent deployments, unique safety properties, or comparative effectiveness.

External validity is limited by synthetic records, local experiments, and historical reference tests. Neither production diversity nor adversarial coverage is represented adequately to support broad deployment claims. The relation between benchmark memory paths and a full application remains partly unresolved in the parity records. Missing metadata stays unknown; a deterministic timestamp, artifact publication date, and execution date are not interchangeable.

Finally, provenance validity and scientific validity are different. The ledger offers exact record identity and admitted mappings, but its trust boundary still depends on source capture, admission policy, and the operator-controlled environment. A private artifact hash cannot enable an outside reader to reproduce unavailable data. The author is associated with the research program, and the current review is AI-assisted preparation rather than independent external peer review.

8 Limitations and reproducibility

The full ADI contract has not been demonstrated by the cited benchmark set. In particular, the evidence does not establish complete mediation of all effect channels, recovery correctness across crashes, resistance to privileged administrator compromise, continuous temporal enforcement, or preservation of state through a live

model replacement. These are outstanding requirements, not features inferred from the architectural description.

Proprietary implementation details, private source, secret schemas, and production topology are excluded. The packet includes sanitized claim mappings, source artifact fingerprints, review notes, and this manuscript, but not the raw private benchmark corpus or a runnable implementation. It therefore supports traceability for authorized inspection more strongly than independent reproduction. A reader without artifact access cannot validate the empirical results from hashes alone.

An independently reproducible program could publish a behavioral specification, synthetic task generator, bounded reference monitor, versioned policy fixtures, adversarial tests, recorded environment, and raw permitted results after a separate disclosure decision. A clean reference implementation could evaluate the conceptual contract without exposing private mechanisms. None of those future materials is claimed to have been released or validated here.

There is also a scope cost. More external controls can increase latency, operational complexity, and false rejection. A fail-closed path can preserve an authority invariant while reducing availability. Centralized enforcement can concentrate failure modes; distributed enforcement can introduce inconsistency. These tradeoffs require measurement against credible baselines, including optimized retrieval and policy-enforced agent systems. The present paper does not claim that more control layers always improve outcomes.

9 Falsifiable predictions and research agenda

Table 3 separates architectural conformance requirements from empirical hypotheses. A system that fails a required invariant is not made conformant by good average task scores. Conversely, conformance alone does not confirm hypotheses about usefulness or efficiency.

Table 3. Proposed tests, with explicit failure criteria. No row is a completed experiment.

Question	Proposed test	Disconfirming observation
Authority invariance	Replay identical proposals with fixed authenticated state and policy while varying inference origin	A decision changes without a declared policy dependency
Complete mediation	Inject disallowed proposals at every protected effect boundary, including memory and release	Any prohibited effect succeeds through an unmediated path
Memory lifecycle correctness	Advance time and revocation state during recall, restart, and retry tests	An expired or out-of-scope record is released contrary to policy
Worker continuity	Replace inference provider mid-workflow while retaining external principal and state	Identity, rights, or accepted history changes without an authorized transition
Bounded substitution utility	Compare model configurations on unseen tasks, matched budgets, and optimized context baselines	The prespecified quality margin or required abstention contract fails
Evidence preservation	Replay failures, corrections, duplicate records, and partial runs	History disappears or one experiment is counted as multiple independent results
Concurrent transition integrity	Race conflicting actions against a revocation or shared resource limit	A protected effect commits under stale authority or violates the specified order

For authority invariance, the decision trace should record all declared inputs, including model identity if policy uses it. Varying the proposal at the same time would test a different property. For complete mediation, the evaluation should include attacks at application adapters and alternate effect channels, not just a list of prompts that the model refuses. Report the attack space and test coverage rather than claiming universal prevention from zero observed escapes. AgentDojo provides an adjacent attack-and-utility evaluation setting [23]; the recent Ajar preprint motivates a separate measure of unnecessary privilege that remains available even when tested attacks do not succeed [24]. Neither supplies evidence that ADI has passed those evaluations.

For substitution utility, an independent protocol should freeze tasks, scorers, equivalence margins, workload clusters, order, budgets, and failure handling before the measured runs. Quality and resource use should be reported jointly, including setup and failed trials. A credible baseline should receive competent retrieval and comparable output budgets. The experiment should distinguish a model-size association from a causal effect of context preparation.

For concurrent transitions, preregister the consistency contract and inject conflicts between authorization and commit. A sequential replay can pass while a race violates the rule. Record accepted effects and their state versions, including interruptions and duplicate requests; test that the observed order satisfies the stated contract. This is a proposed conformance test, not evidence of current race freedom.

For memory lifecycle and continuity, the program should test time-dependent validity after admission, scope changes, conflicting updates, service interruption, replay, and rollback. State must be compared at defined checkpoints. Successful recall after a restart is a narrower property than long-term worker continuity; neither can be inferred from naming a persistent principal. A later external study should reproduce both accepted and held outcomes where its disclosure rights permit.

A practical sequence is to publish a disclosure-safe behavioral contract, construct a reference implementation with explicit assumptions, and run conformance and ablation studies before expanding claims about generality. A separate study should compare lifecycle coverage and utility with existing governed-agent systems. The umbrella terminology earns its usefulness only if it improves specification, measurement, or diagnosis beyond those already available approaches.

10 Conclusion

ADI names an architectural approach in which probabilistic models perform bounded inference while external deterministic systems govern authority-bearing decisions. Its distinguishing emphasis is a consistent lifecycle contract across identity, memory admission, execution, verification, and release. Its antecedents are substantial; the proposal is an explicit synthesis and research agenda, not a claim to have invented external verification or access control.

The available ledger supports bounded model-configuration comparisons, historical memory observations, and incomplete parity findings. It also preserves negative results and excludes records with unresolved provenance. It does not establish universal model equivalence, full architectural conformance, or independent reproducibility. The next contribution must be a falsifiable and reproducible demonstration of the stated authority properties. Probabilistic reasoning. Deterministic authority.

Data availability and declarations

The empirical records are private. EVIDENCE_MAP.md identifies the exact admitted records and claim tags; CLAIMS.md separates observations, derivations, definitions, and hypotheses. Full raw artifacts and proprietary implementation are not included. The companion manuscript reuses the same underlying lineage and is not independent corroboration. No DOI, public dataset, public code release, submission, or external peer-review acceptance is claimed.

The manuscript was prepared with AI assistance for literature research, evidence mapping, drafting, and adversarial review. Installed department stages provided governed source preparation; substantive drafting and reviews were completed in the authoring environment. These are not independent human reviews or legal clearance. The author retains responsibility for factual accuracy, authorship, disclosure, and any later release decision.

References

All web references were checked on 9 October 2026. Preprints and living documentation are identified as such.

- [1] James P. Anderson. 1972. Computer Security Technology Planning Study, Volume I. ESD-TR-73-51. Electronic Systems Division, Air Force Systems Command. <https://csrc.nist.gov/files/pubs/conference/1998/10/08/proceedings-of-the-21st-nissc-1998/final/docs/early-cs-papers/ande72a.pdf>
- [2] Jerome H. Saltzer and Michael D. Schroeder. 1975. The Protection of Information in Computer Systems. *Proceedings of the IEEE* 63(9), 1278–1308. <https://web.mit.edu/Saltzer/www/publications/protection/>
- [3] Mark S. Miller, Ka-Ping Yee, and Jonathan Shapiro. 2003. Capability Myths Demolished. Johns Hopkins University Systems Research Laboratory, Technical Report SRL2003-02. <https://srl.cs.jhu.edu/pubs/SRL2003-02.pdf>
- [4] Leslie Lamport. 1994. The Temporal Logic of Actions. *ACM Transactions on Programming Languages and Systems* 16(3), 872–923. <https://lamport.org/pubs/lamport-actions.pdf>
- [5] D. Seto, B. Krogh, L. Sha, and A. Chutinan. 1998. The simplex architecture for safe on-line control system upgrades. *American Control Conference* 6, 3504–3508. <https://doi.org/10.1109/ACC.1998.703255>
- [6] Fred B. Schneider. 2000. Enforceable Security Policies. *ACM Transactions on Information and System Security* 3(1), 30–50. <https://www.cs.cornell.edu/fbs/publications/EnfSecPols.pdf>
- [7] OASIS. 2013. eXtensible Access Control Markup Language (XACML) Version 3.0. OASIS Standard, 22 January 2013. <https://docs.oasis-open.org/xacml/3.0/xacml-3.0-core-spec-os-en.pdf>
- [8] Open Policy Agent contributors. Open Policy Agent documentation. Accessed 9 October 2026. <https://www.openpolicyagent.org/docs>
- [9] Joseph W. Cutler et al. 2024. Cedar: A New Language for Expressive, Fast, Safe, and Analyzable Authorization (Extended Version). arXiv:2403.04651v2. <https://arxiv.org/abs/2403.04651v2>
- [10] Subbarao Kambhampati et al. 2024. Position: LLMs Can't Plan, But Can Help Planning in LLM-Modulo Frameworks. *ICML, PMLR* 235, 22895–22907. <https://proceedings.mlr.press/v235/kambhampati24a.html>
- [11] Edoardo Debenedetti et al. 2025. Defeating Prompt Injections by Design. arXiv:2503.18813v2. <https://arxiv.org/abs/2503.18813v2>
- [12] Haoyu Wang, Christopher M. Poskitt, and Jun Sun. 2026. AgentSpec: Customizable Runtime Enforcement for Safe and Reliable LLM Agents. *ICSE*, 2938–2950. <https://doi.org/10.1145/3744916.3764546> ; author version: <https://arxiv.org/abs/2503.18666v3>
- [13] Tianneng Shi et al. 2026. Progent: Securing AI Agents with Privilege Control. arXiv:2504.11703v3, revised 14 May 2026; initial preprint 2025. <https://arxiv.org/abs/2504.11703v3>
- [14] Luca Beurer-Kellner et al. 2025. Design Patterns for Securing LLM Agents against Prompt Injections. arXiv:2506.08837v3. <https://arxiv.org/abs/2506.08837v3>
- [15] Robin Manhaeve, Sebastijan Dumančić, Angelika Kimmig, Thomas Demeester, and Luc De Raedt. 2018. DeepProbLog: Neural Probabilistic Logic Programming. *NeurIPS* 31. https://proceedings.neurips.cc/paper_files/paper/2018/hash/dc5d637ed5e62c36ecb73b654b05ba2a-Abstract.html
- [16] Patrick Lewis et al. 2020. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. *NeurIPS* 33. https://proceedings.neurips.cc/paper_files/paper/2020/hash/6b493230205f780e1bc26945df7481e5-Abstract.html
- [17] Charles Packer et al. 2023. MemGPT: Towards LLMs as Operating Systems. arXiv:2310.08560. <https://arxiv.org/abs/2310.08560>
- [18] Shunyu Yao et al. 2023. ReAct: Synergizing Reasoning and Acting in Language Models. *ICLR*; initial preprint 2022. <https://arxiv.org/abs/2210.03629v3>
- [19] Branden Laskowski. 2026. Externalized Intelligence and Model Substitution. Manuscript v1.0, 8 October 2026. Unpublished private staging. Supporting synthesis of the same research lineage, not independent replication; no public access link or DOI.

- [20] Cortex Research & Publication Department. 2026. Governed Research Evidence Ledger, ADI scope, snapshot retrieved 9 October 2026. Private admitted research records. Exact identities, tags, source hashes, and inspection limits are in EVIDENCE_MAP.md; no public access link or DOI.
- [21] Kai Mei et al. 2025. AIOS: LLM Agent Operating System. COLM; initial preprint 2024. <https://arxiv.org/abs/2403.16971v5>
- [22] Gerwin Klein et al. 2009. seL4: Formal Verification of an OS Kernel. SOSP, 207–220. <https://sel4.org/Research/pdfs/sel4-formal-verification-os-kernel.pdf>
- [23] Edoardo Debenedetti et al. 2024. AgentDojo: A Dynamic Environment to Evaluate Prompt Injection Attacks and Defenses for LLM Agents. NeurIPS Datasets and Benchmarks. <https://arxiv.org/abs/2406.13352>
- [24] Reshabh K Sharma, Linxi Jiang, Shuo Chen, and Zhiqiang Lin. 2026. Ajar: Measuring Open Privilege in Agent Defenses. arXiv:2609.26900v1, 22 September 2026. Preprint. <https://arxiv.org/abs/2609.26900v1>